

Lab Assignment 2.1

2303A51887

Task 1: Statistical Summary for Survey Data

Scenario:

- You are a data analyst intern working with survey responses stored as numerical lists.

Task:

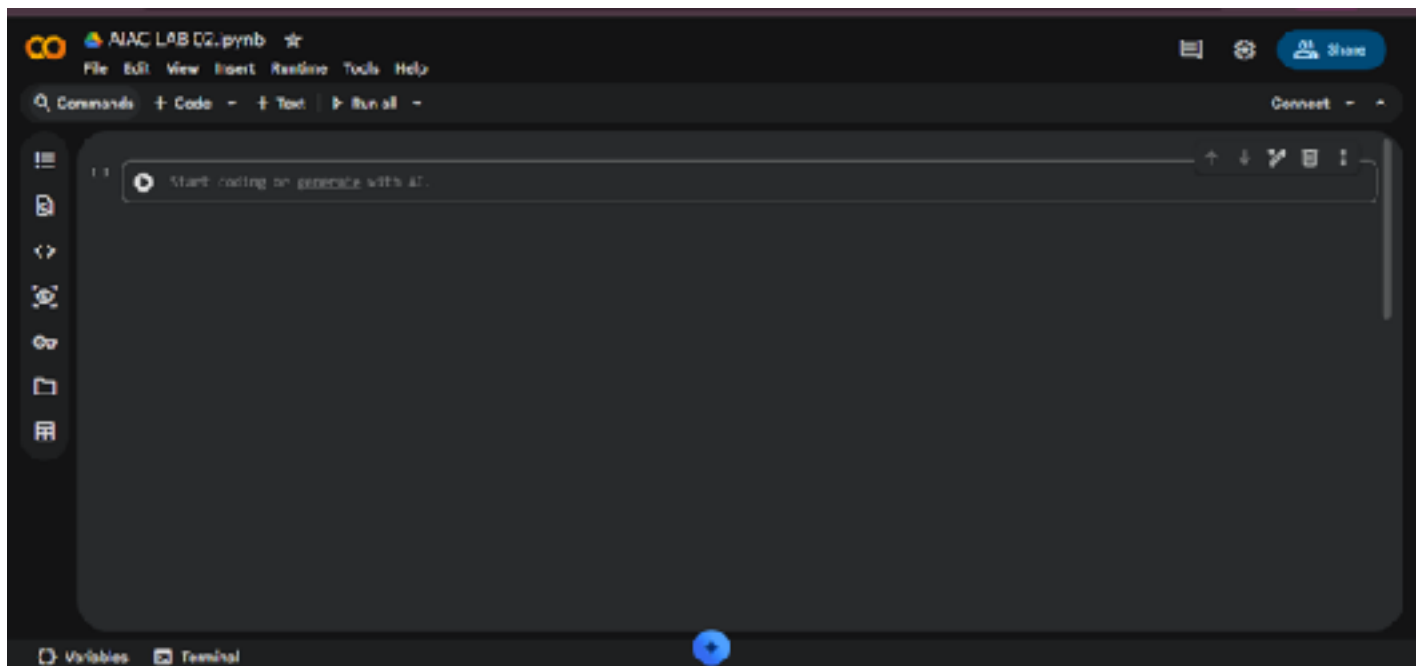
- Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values.

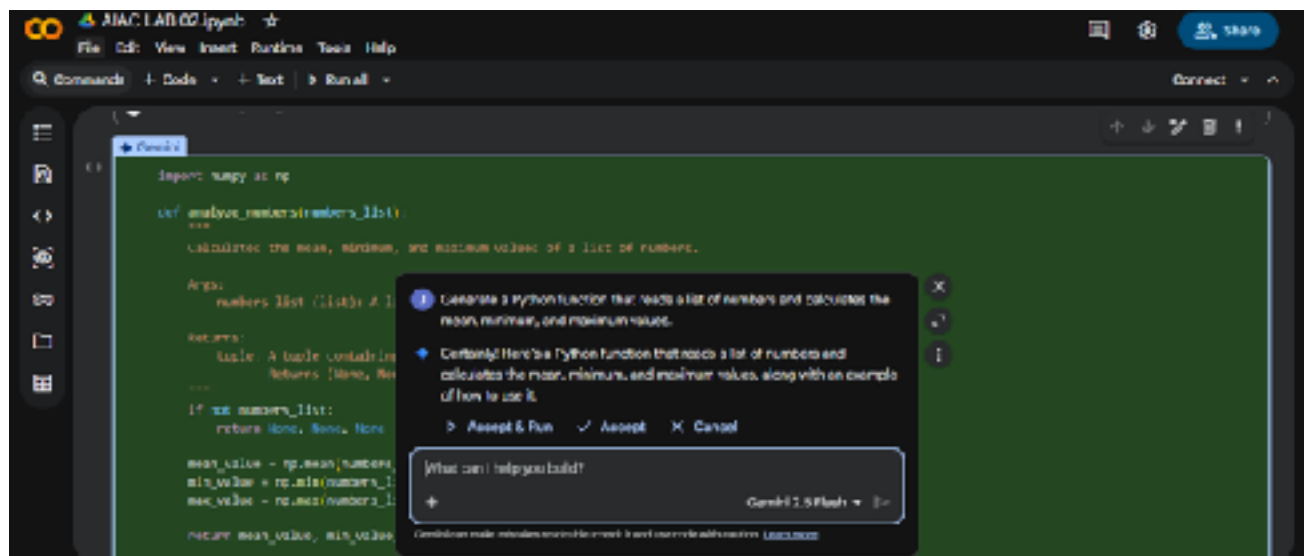
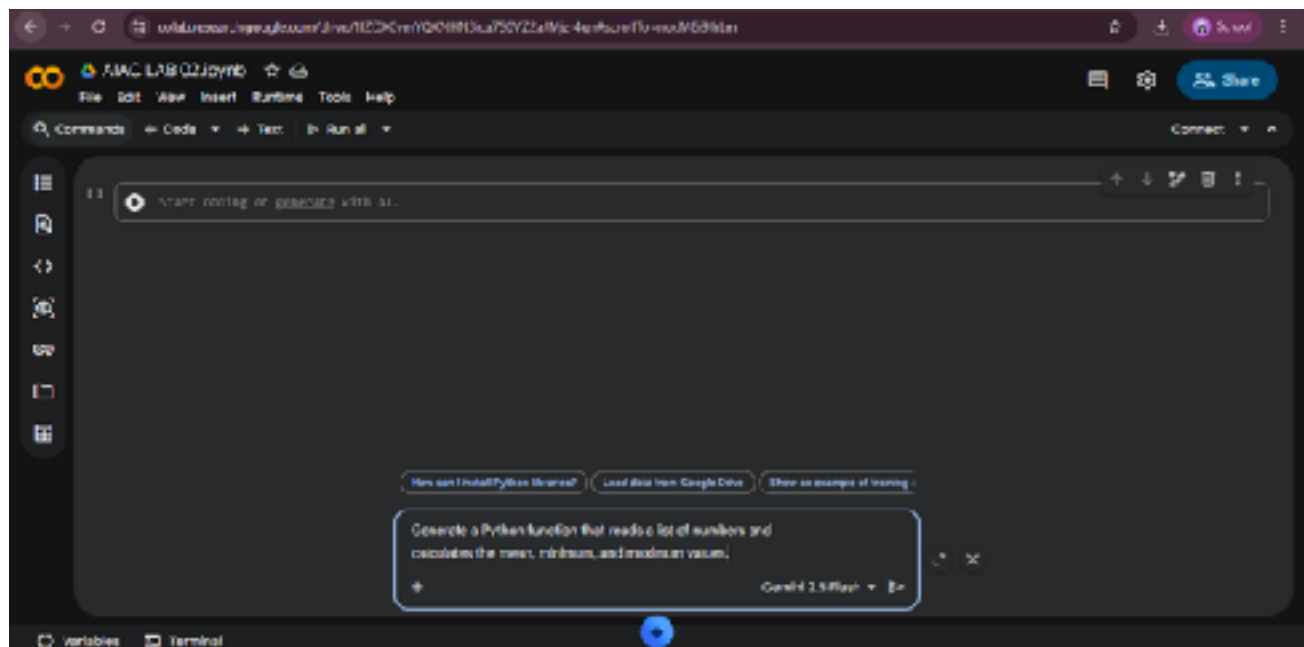
Expected Output:

- Correct Python function
- Output shown in Colab
- Screenshot of Gemini prompt and result

Google Colab Workspace Screenshots:

```
Generate a Python function that reads a list of numbers and
calculates the mean, minimum, and maximum values.
```





AIAC LAB 02.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

11

✓

10

```
import numpy as np

def analyze_numbers(numbers_list):
    """
    Calculates the mean, minimum, and maximum values of a list of numbers.

    Args:
        numbers_list (list): A list of numerical values.

    Returns:
        tuple: A tuple containing the mean, minimum, and maximum values.
        Returns (None, None, None) if the list is empty.
    """
    if not numbers_list:
        return None, None, None

    mean_value = np.mean(numbers_list)
    min_value = np.min(numbers_list)
    max_value = np.max(numbers_list)

    return mean_value, min_value, max_value

# Example usage:
my_numbers = [10, 20, 30, 40, 50, 5, 25, 35]
mean, minimum, maximum = analyze_numbers(my_numbers)

print(f"List of numbers: {my_numbers}")
```

AIAC LAB 02.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

11

✓

10

```
mean_value = np.mean(numbers_list)
min_value = np.min(numbers_list)
max_value = np.max(numbers_list)

return mean_value, min_value, max_value

# Example usage:
my_numbers = [10, 20, 30, 40, 50, 5, 25, 35]
mean, minimum, maximum = analyze_numbers(my_numbers)

print(f"List of numbers: {my_numbers}")
print(f"Mean: {mean}")
print(f"Minimum: {minimum}")
print(f"Maximum: {maximum}")

# Example with an empty list:
empty_list = []
mean_empty, min_empty, max_empty = analyze_numbers(empty_list)
print(f"Empty list: {empty_list}")
print(f"Mean: {mean_empty}, Minimum: {min_empty}, Maximum: {max_empty}")
```

11

✓

10

```
List of numbers: [10, 20, 30, 40, 50, 5, 25, 35]
Mean: 25.375
Minimum: 5
Maximum: 50

Empty list: []
Mean: None, Minimum: None, Maximum: None
```

Task 2: Armstrong Number – AI Comparison

Scenario:

- You are evaluating AI tools for numeric validation logic.

Task:

- Generate an Armstrong number checker using Gemini and GitHub Copilot.
- Compare their outputs, logic style, and clarity.

Expected Output:

- Side-by-side comparison table
- Screenshots of prompts and generated code

Google Colab Workspace Screenshots:

```
Write a Python function to check if a number is an Armstrong
number
```



Commands + Code + Text ▶ Run all

[1]
✓ 0s

```
mean_empty, min_empty, max_empty = analyze_numbers(empty_list)
print(f"\nEmpty list: {empty_list}")
print(f"Mean: {mean_empty}, Minimum: {min_empty}, Maximum: {max_empty}")
```

▼

```
+++ List of numbers: [10, 20, 30, 40, 50, 5, 25, 35]
```

```
Mean: 26.875
```

```
Minimum: 5
```

```
Maximum: 50
```

```
Empty list: []
```

```
Mean: None, Minimum: None, Maximum: None
```

OR HOW TO USE IT:

The Python function `analyze_numbers` has successfully calculated the mean, minimum, and maximum values for your list `[10, 20, 30, 40, 50, 5, 25, 35]`. The results are:

- Mean: 26.875
- Minimum: 5

[Add standard deviation calculation to analyze_numbers function](#)[Explain the np.mean and np](#)

Write a Python function to check if a number is an Armstrong number

+

Gemini 2.5 Flash ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Variables

Terminal

Dfvhbn

Tgrhnmnb

```
week 2 > armstrong.py > ...
1  def is_armstrong(n):
5      Args:
6          n: An integer to check
7
8      Returns:
9          True if n is an Armstrong number, False otherwise
10         ....
11         # Convert to string to get number of digits
12         digits = str(abs(n))
13         num_digits = len(digits)
14
15         # Calculate sum of each digit raised to the power of num_digits
16         armstrong_sum = sum(int(digit) ** num_digits for digit in digits)
17
18         return armstrong_sum == abs(n)
19
20
21     # test cases
22     if __name__ == "__main__":
23         test_numbers = [153, 370, 371, 407, 1634, 123, 100]
24
25         for num in test_numbers:
26             print(f"{num} is Armstrong: {is_armstrong(num)}")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

153 is Armstrong: True
370 is Armstrong: True
371 is Armstrong: True
407 is Armstrong: True
1634 is Armstrong: True
123 is Armstrong: False
100 is Armstrong: False
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task 3: Leap Year Validation Using Cursor AI

Scenario:

- You are validating a calendar module for a backend system.

Task:

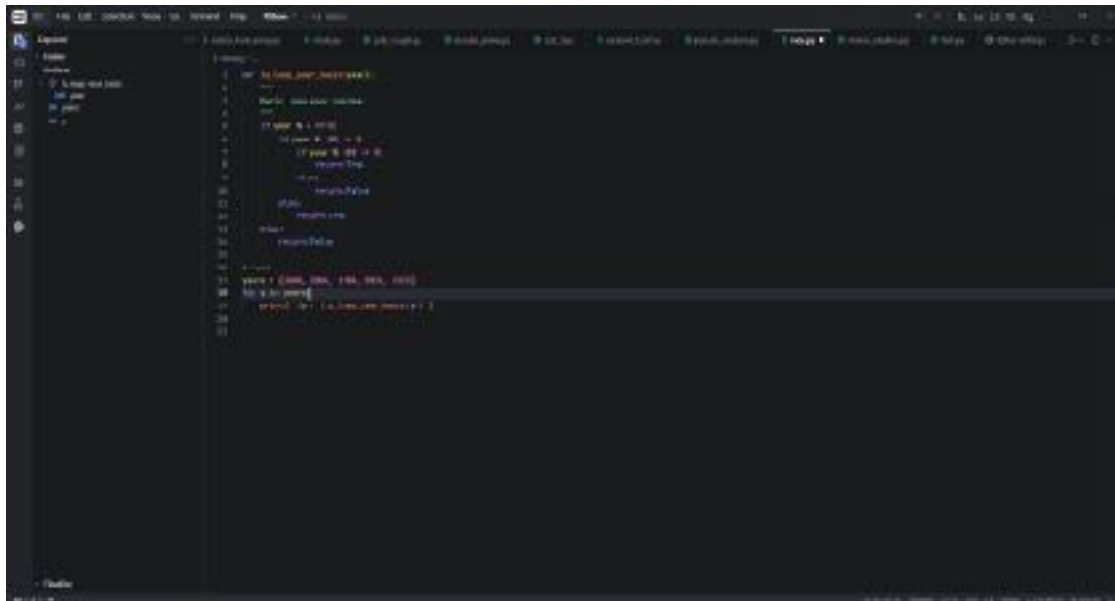
- Use Cursor AI to generate a Python program that checks whether a given year is a leap year.
- Use at least two different prompts and observe changes in code.

Expected Output:

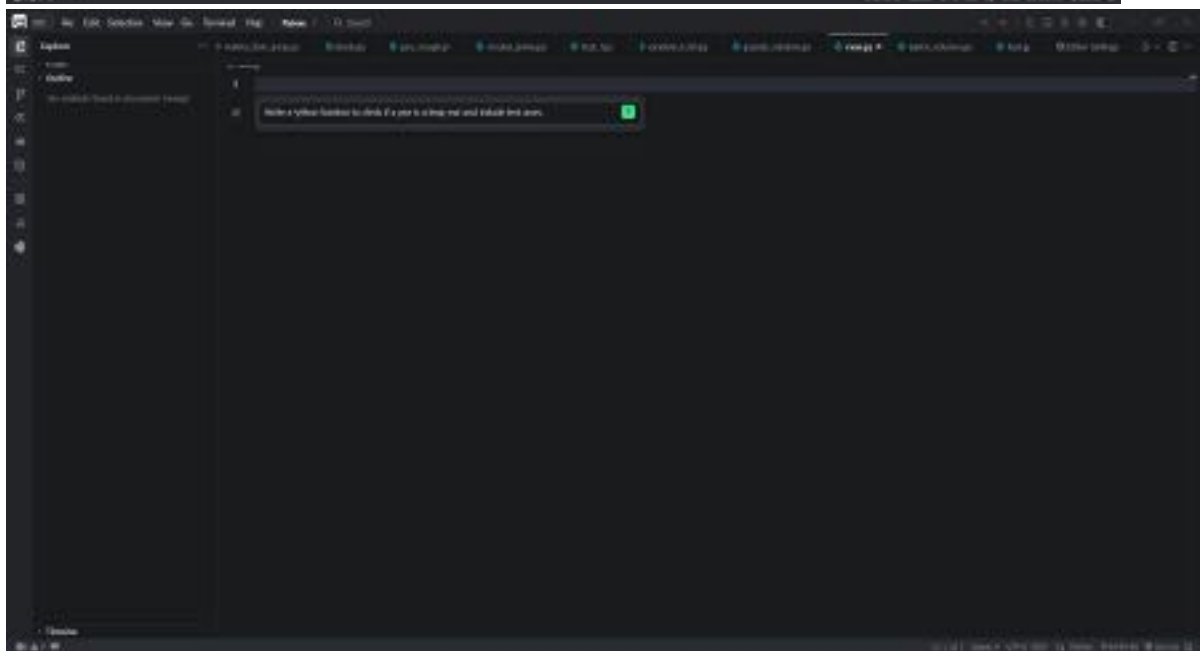
- Two versions of code
- Sample inputs/outputs
- Brief comparison

Instruction Given:

Write a Python function to check if a year is a leap year and include test cases.



```
def is_leap_year(year):  
    """Check if a year is a leap year.  
    A leap year is a year that is evenly divisible by 4, but not by 100, unless it is also divisible by 400.  
    """  
    if year % 4 == 0:  
        if year % 100 == 0:  
            if year % 400 == 0:  
                return True  
            else:  
                return False  
        else:  
            return True  
    else:  
        return False  
  
# Test cases  
year = 2000  
print(is_leap_year(year))  
  
year = 2001  
print(is_leap_year(year))  
  
year = 1900  
print(is_leap_year(year))  
  
year = 2024  
print(is_leap_year(year))  
  
year = 2023  
print(is_leap_year(year))
```



```
# Write a Python function to check if a year is a leap year and include test cases.
```

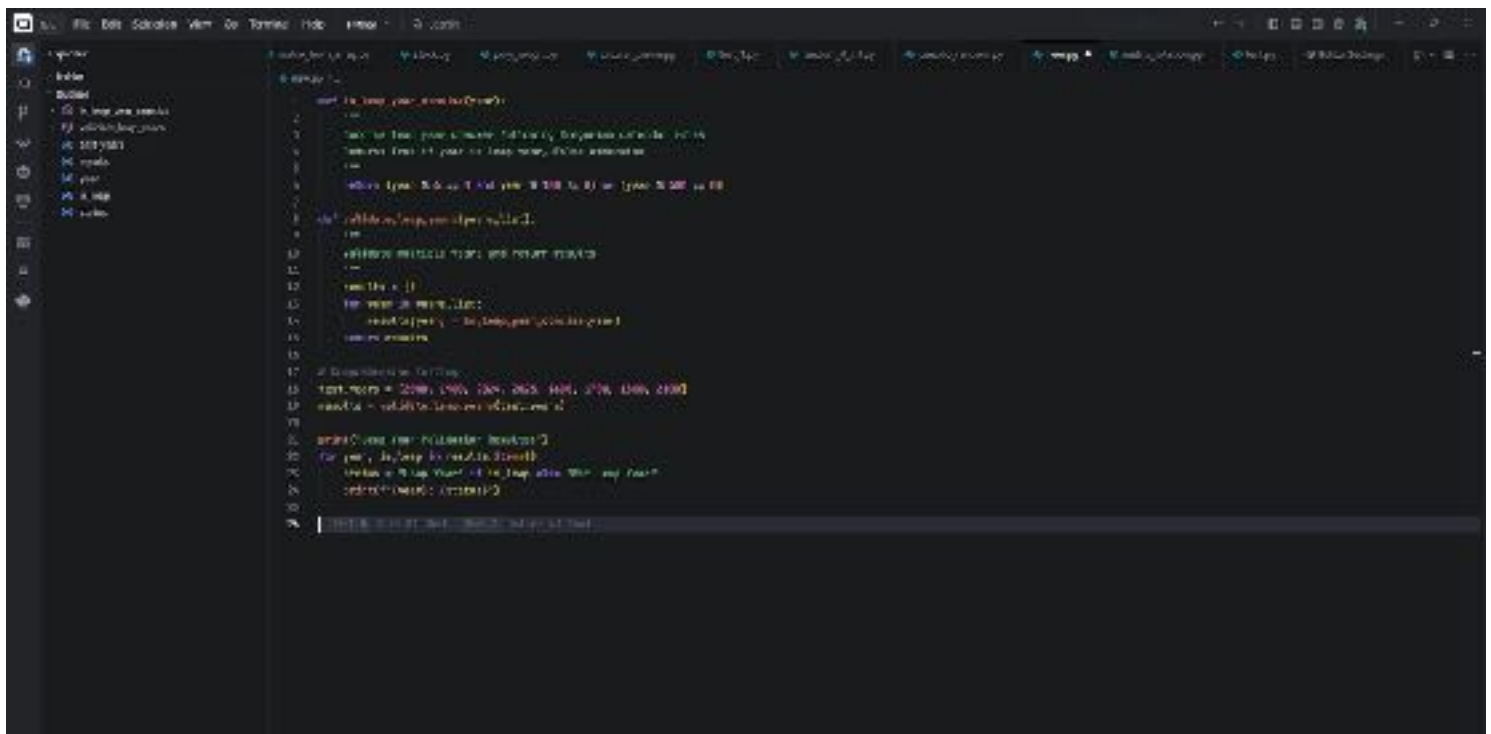
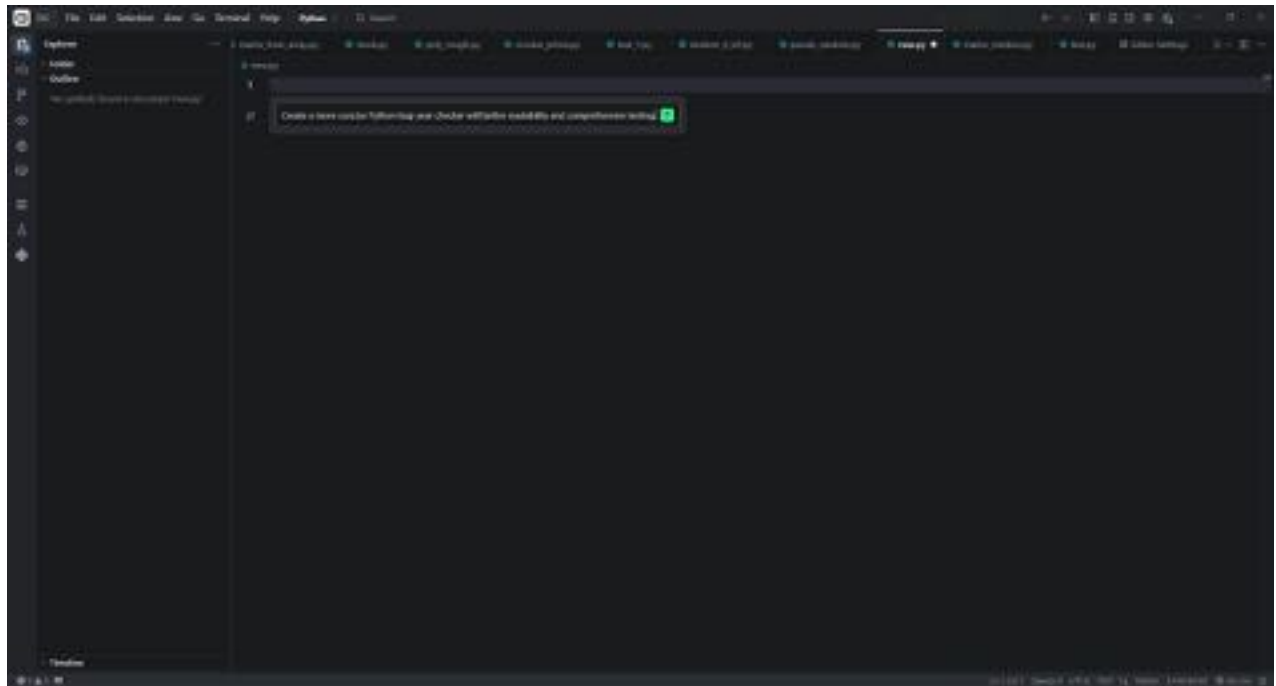
Code Output:

```
2000: True  
2001: True  
1900: False  
2024: True  
2023: False
```

Version - 2:

Instruction Given:

Create a more concise Python leap year checker with better readability and comprehensive testing.



Code Output:

```
Leap Year Validation Results:
2000: Leap Year
1400: Not Leap Year
2024: Leap Year
2023: Not Leap Year
1600: Leap Year
1700: Not Leap Year
1800: Not Leap Year
2100: Not Leap Year
```

Brief comparison:

Version 1 (Basic)	Version 2 (Concise)
Uses nested if-else statements	Single return statement with logical operators

More lines of code (12 lines)	Fewer lines (5 lines for main function)
Easier for beginners to understand	Follows Python style guide
Step-by-step logic	Combined logical conditions
Includes basic testing	Includes comprehensive testing function

Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

Scenario:

- Company policy requires developers to write logic before using AI.

Task:

- Write a Python program that calculates the sum of odd and even numbers in a tuple, then refactor it using any AI tool.

Expected Output:

- Original code
- Refactored code
- Explanation of improvements

Original Code written by me:

```
def
even_odd_sum_tuple
(num): even_sum =
0
odd_sum = 0

    for i in
range(1, num+1):
    if i%2 == 0:
        even_su
m += i else:
        odd_sum += i

return even_sum, odd_sum
```

Refactored code:

Instructions given:

Refactor the code **in** order to make it handle input validation, readability **and** handle edge cases.

```
def even_odd_sum_tuple(n: int) ->
tuple[int, int]: """
Calculate the sum of even and odd numbers from 1 to
n.
```

Parameters:

n (int): The upper limit (must be a positive integer).

Returns:

```
tuple[int, int]: (sum_of_even_numbers,
sum_of_odd_numbers)
"""
```

```
if n < 1:
```

```
    return (0, 0) # No positive
```

```
range to sum even_sum, odd_sum =
```

```
0, 0
```

```
    for i in
range(1, n +
1): if i % 2 ==
0:
        even_s
um += i else:
        odd_sum
```

Explanation of improvements:

Input validation

- Making sure the user enters a valid integer.
- Handling negative numbers gracefully (return (0,0) or raise an error). Readability
- Using clear variable names.
- Adding docstrings and comments.

Edge cases

- If input is 0, both sums should be 0.
- If input is negative, we can either reject it or compute sums up to that number (here I'll reject it for clarity).